

Aula 9 — Métodos de suavização

KNN-regressão; kernel smoothing; maldição da dimensionalidade

Prof. Dr. João Paulo Ferreira de Mello

5 de outubro de 2026

Ciências de Dados

Fatec Rubens Lara – Baixada Santista

O que vamos ver hoje

- **De onde viemos:** nos Blocos I–II todo modelo tinha uma *forma* escolhida de antemão — reta, polinômio, logística — e aprendemos a avaliá-lo (Aulas 6–7) e a freá-lo (Aula 8). A P1 fechou essa etapa.
- **O novo:** hoje viramos a chave. Modelos que **não supõem forma**: para prever em um ponto, olham só a *vizinhança* dele. Dois jeitos de fazer isso — **KNN-regressão** e **kernel** —, cada um com seu botão de flexibilidade; e o limite dos dois: a **maldição da dimensionalidade**.
- **Para onde vai:** a Aula 10 dá a esses suavizadores controle e vários preditores (*splines* e GAM); depois, árvores.

O fio da aula: a imobiliária de novo — **150 imóveis, um só preditor** (a área) e um preço que sobe em **degraus**.

Por que sair do paramétrico

Os dados de hoje: um preditor, 150 imóveis, dois degraus

```
head(dados, 4)      # 150 imóveis: area (m2) e preco (mil R$)

#>   area preco
#> 1    40   156
#> 2    41   233
#> 3    41   197
#> 4    41   172
```

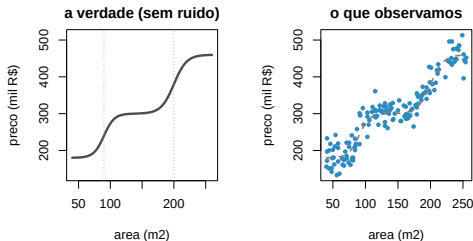
Como os dados são *simulados*, sabemos a verdade — e o modelo não:

$$\text{preço} = 180 + 120 \sigma\left(\frac{\text{área} - 90}{8}\right) + 160 \sigma\left(\frac{\text{área} - 200}{10}\right) + \varepsilon, \quad \varepsilon \sim N(0, 25^2),$$

- $\sigma(z) = 1/(1 + e^{-z})$ é a curva em S da Aula 5 (a logística): cada termo é um **degrau suave** — o mercado “muda de faixa” perto de 90 m² e de 200 m²;
- o ruído tem desvio 25 mil R\$: é o piso irreduzível;
- **nenhum polinômio é essa função**. É esse o ponto da aula.

A verdade sem ruído — e o que os dados mostram

```
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
curve(f, 40, 260, lwd = 3, col = "gray30", ylim = range(preco),
      main = "a verdade (sem ruído)", xlab = "area (m2)", ylab = "preco (mil R$)")
abline(v = c(90, 200), lty = 3, col = "gray60") # os dois degraus
plot(area, preco, pch = 16, col = "cazul", cex = 0.8, main = "o que observamos",
      xlab = "area (m2)", ylab = "preco (mil R$)")
curve(f, add = TRUE, col = "gray50", lty = 2, lwd = 2) # a verdade, tracejada
```



Leia a figura: sem ruído, o preço tem três patamares (≈ 180 , ≈ 300 e ≈ 460 mil) ligados por dois degraus suaves, em 90 e em 200 m². Com o ruído, os degraus ainda aparecem, mas a forma exata some. É essa curva — tracejada daqui em diante — que os métodos de hoje tentam reencontrar.

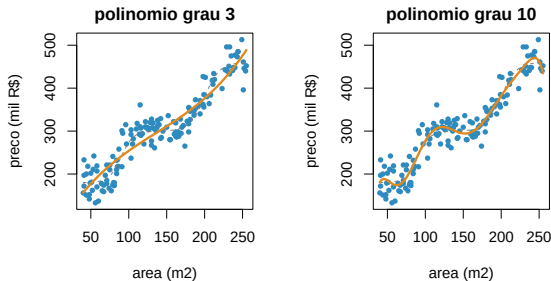
O teto do polinômio: girando o botão da Aula 6 até o fim

Na Aula 6 o grau do polinômio era o **botão de flexibilidade**. Vamos girá-lo: grau 3 e grau 10, contra a verdade (tracejada).

```
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
for (g in c(3, 10)) {
  m <- lm(preco ~ poly(area, g), data = dados)
  plot(area, preco, pch = 16, col = "blue", cex = 0.8,
       main = paste("polinomio grau", g),
       xlab = "area (m2)", ylab = "preco (mil R$)")
  lines(grade, f(grade), col = "gray50", lty = 2)           # a verdade
  lines(grade, predict(m, data.frame(area = grade)),
       col = "red", lwd = 2.5)                             # o ajuste
}
```

Para medir “chegou perto?” sem depender de uma amostra de teste, usamos hoje a **distância até a verdade**: o RMSE entre a curva ajustada e f , nos 300 pontos da grade. Só é possível porque simulamos — na vida real, o substituto é a validação cruzada.

Grau 3 erra a forma; grau 10 acerta a forma e erra as pontas



Leia a figura: o grau 3 atravessa o patamar de 100 a 180 m² como se fosse rampa — uma forma só não faz dois degraus. O grau 10 acompanha os degraus, mas na ponta direita *vira para baixo* onde os dados não viram. Distância até a verdade — grau 1: 21,0; grau 3: 18,8; grau 6: 8,1; grau 10: 7,9.

Observação

O polinômio é **global**: um só conjunto de coeficientes serve para 40 m² e para 250 m² ao mesmo tempo. Para fazer o degrau ele precisa de grau alto — e grau alto oscila nas pontas. A saída: decidir **localmente**.

A ideia: para prever em um ponto, olhe só a vizinhança dele

Lembrete da Aula 1. *Paramétrico*: supor a forma de f e estimar poucos parâmetros. *Não-paramétrico*: deixar os dados moldarem a curva. Até hoje só fizemos o primeiro. Agora, o segundo.

“Quanto vale um imóvel de 120 m²?” tem uma resposta ingênua — e boa: **olhe os imóveis de área parecida e tire a média**. Os dois métodos de hoje são variações dessa frase:

	quem é vizinho	com que peso	o botão
KNN-regressão kernel	os k mais próximos todos	igual para todos decai com a distância	k , o número de vizinhos h , a largura da janela

- O botão de flexibilidade muda de nome — k ou h — mas quem o gira continua sendo a **validação cruzada** (Aula 7).
- “Sem supor a forma” não é “sem hipóteses”: supomos que f é **suave** — vizinhos em área têm preços parecidos.

KNN-regressão

KNN-regressão: a média dos k vizinhos mais próximos

Definição

KNN-regressão (*k-nearest neighbours*, k vizinhos mais próximos). Para prever a resposta em um ponto x_0 , seja $N_k(x_0)$ o conjunto das k observações cujo preditor está mais perto de x_0 . Então

$$\hat{f}(x_0) = \frac{1}{k} \sum_{i \in N_k(x_0)} y_i.$$

entrada: dados $(x(i), y(i))$, $i = 1..n$; um ponto x_0 ; um numero k

1. para cada i , calcule a distancia $d(i) = |x(i) - x_0|$
2. ordene as observacoes por $d(i)$; as k primeiras formam $N(x_0)$
3. devolva a media dos $y(i)$ com i em $N(x_0)$

Em palavras: não há coeficientes nem “ajuste” — o modelo é a base de dados. Treinar custa zero; prever custa uma ordenação por ponto. E k é o único botão: $k = 1$ copia o vizinho mais próximo; $k = n$ devolve a média geral para todo mundo.

Na mão: um imóvel de 120 m², com $k = 5$

```
x0 <- 120; k <- 5
viz <- order(abs(area - x0))[1:k]      # os 5 indices mais proximos de x0
dados[viz, ]

#>      area preco
#> 62  120    270
#> 61  118    294
#> 63  122    310
#> 64  122    322
#> 65  123    308

mean(preco[viz])                      # a predicao do KNN

#> [1] 300.8
```

Leia a saída: os cinco vizinhos têm de 118 a 123 m²; a média dos preços deles, 300,8, é a predição. A verdade em 120 m² é $f(120) = 297,3$ — erro de 3,5 mil, com ruído de 25. Nenhuma fórmula foi suposta: só distância e média.

O botão k : 3 vizinhos ou 60?

A conta da página anterior, empacotada — e aplicada a cada ponto da grade, com $k = 3$ e com $k = 60$:

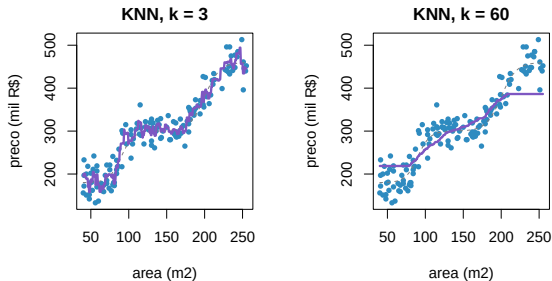
```
knn_reg <- function(x0, x, y, k) {  
  viz <- order(abs(x - x0))[1:k]  
  mean(y[viz])  
}
```

```
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")  
for (k in c(3, 60)) {  
  plot(area, preco, pch = 16, col = "blue", cex = 0.8,  
        main = paste("KNN, k =", k),  
        xlab = "area (m2)", ylab = "preco (mil R$)")  
  lines(grade, f(grade), col = "gray50", lty = 2)  
  lines(grade, sapply(grade, knn_reg, x = area, y = preco, k = k),  
        col = "red", lwd = 2.5)  
}
```

`sapply(grade, knn_reg, ...)` chama a função uma vez para cada um dos 300 pontos da grade.

Distância até a verdade — $k = 1$: 27,1; $k = 3$: 14,6; $k = 10$: 8,3; $k = 20$: **6,9**; $k = 40$: 15,3; $k = 80$: 48,5.

k pequeno serrilha; k grande achata



Leia a figura: com $k = 3$ a curva *serrilha* — segue cada imóvel, inclusive o ruído (variância alta). Com $k = 60$ ela fica lisa, mas achata os degraus e vira *patamar* nas pontas: um imóvel de 250 m² é previsto pela média dos de 162 a 255 m² (viés alto).

Observação

O melhor k (6,9) chega mais perto da verdade que o melhor grau (7,9) — sem supor forma. Mas foi escolhido *olhando a verdade*, que ninguém tem: na prática, quem escolhe é a validação cruzada.

Escolhendo k por validação cruzada (Aula 7)

As mesmas 5 dobras da Aula 7; em cada rodada, os vizinhos são procurados **só no treino** — a dobra de validação nunca vê a si mesma:

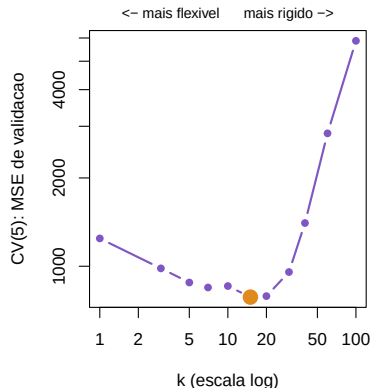
```
set.seed(1); dobra <- sample(rep(1:5, length = n))      # 5 dobras
cv_knn <- function(k) mean(sapply(1:5, function(j) {
  tr <- dobra != j; va <- dobra == j
  pred <- sapply(area[va], knn_reg, x = area[tr], y = preco[tr], k = k)
  mse(pred, preco[va])
})))
ks <- c(1, 3, 5, 7, 10, 15, 20, 30, 40, 60, 100)
cvk <- sapply(ks, cv_knn)
setNames(round(cvk), ks)                                # CV(5) por k

#>      1      3      5      7     10     15     20     30     40     60    100
#> 1244   983   880   846   856   785   791   956  1404  2838  5869
```

Leia a saída: o erro cai de 1244 ($k = 1$) até **785** ($k = 15$), e volta a subir — 5 869 em $k = 100$, quando a “vizinhança” já é dois terços da base. O piso é $25^2 = 625$.

O mesmo U — espelhado

```
par(mar = c(4,4,2.6,1), pty = "s")
plot(ks, cvk, type = "b", log = "xy", pch = 16,
     col = croxo, lwd = 2,
     xlab = "k (escala log)",
     ylab = "CV(5): MSE de validacao")
abline(h = 625, lty = 2, col = "gray50") # piso
points(15, cvk[ks == 15], pch = 16,
       cex = 2, col = claranja)
mtext("<- mais flexivel      mais rigido ->",
      side = 3, line = 0.4, cex = 0.8)
```



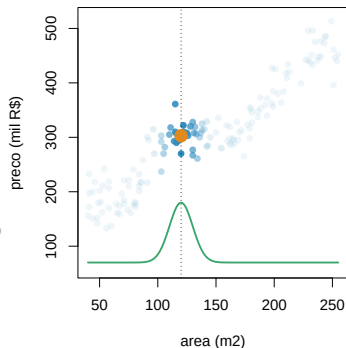
Leia a figura: é o U das Aulas 6–7, mas aqui a flexibilidade cresce *para a esquerda*: k pequeno é o modelo flexível (variância), k grande é o rígido (viés). O fundo está em $k = 15$; $k = 20$ (791) empata — e, na dúvida, prefira o mais rígido.

Kernel smoothing

Pesos que decaem com a distância

No KNN, o 15º vizinho conta tanto quanto o 1º, e o 16º não conta nada. A alternativa: **todo mundo conta, com peso que diminui com a distância.**

```
x0 <- 120; h <- 10
w <- dnorm((area - x0) / h)      # peso = altura do sino
par(mar = c(4,4,1,1), pty = "s")
plot(area, preco, pch = 16, ylim = c(60, 520),
     col = rgb(0.18, 0.55, 0.75, 0.08 + 0.92 * w / max(w)),
     xlab = "area (m2)", ylab = "preco (mil R$)")
lines(grade, 70 + 110 * dnorm((grade - x0) / h) / dnorm(0),
      col = cverde, lwd = 2)      # o sino, desenhado embaixo
abline(v = x0, lty = 3)
points(x0, sum(w * preco) / sum(w), pch = 16, cex = 2,
       col = claranja)           # a media ponderada
```



Leia a figura: a curva verde, embaixo, é o sino ($h = 10 \text{ m}^2$, centrado em 120 m^2). Cada imóvel pesa a altura do sino acima da sua área: os azuis fortes decidem, os apagados quase não contam. O ponto laranja é a média ponderada — a predição.

A fórmula: a média ponderada pelo sino

Definição

Suavização por núcleo (*kernel smoothing*; estimador de Nadaraya–Watson). Com um *núcleo* K e uma *banda* $h > 0$,

$$\hat{f}(x_0) = \frac{\sum_{i=1}^n K_h(x_0, x_i) y_i}{\sum_{i=1}^n K_h(x_0, x_i)}, \quad K_h(x_0, x) = \exp\left[-\frac{1}{2} \left(\frac{x-x_0}{h}\right)^2\right] \text{ (núcleo gaussiano).}$$

Em palavras: é uma *média ponderada* de todos os y_i ; o denominador faz os pesos somarem 1. Com o núcleo gaussiano, uma observação a **1 banda** de x_0 pesa 61% do que pesa uma em cima de x_0 ; a **2 bandas**, 14%; a **3 bandas**, 1%. Todo mundo entra na conta, mas quem decide é quem está a até umas 3 bandas.

- h é o botão: sino estreito = poucos vizinhos de fato = flexível; sino largo = rígido.
- Outros núcleos (tricúbico, Epanechnikov) mudam pouco. **A forma do sino importa pouco; a largura importa muito.**
- No R, `dnorm((x - x0)/h)` dá esse sino a menos de uma constante — que se cancela na divisão.
- Não confunda com o *kernel* do SVM, no Bloco IV: mesmo nome, outra ideia.

Na mão: o mesmo imóvel de 120 m², banda $h = 10$

```
nw <- function(x0, x, y, h) {  
  w <- dnorm((x - x0) / h)      # o sino: peso de cada imovel  
  sum(w * y) / sum(w)          # a media ponderada  
}  
nw(120, area, preco, h = 10)   # a predicao em 120 m2  
  
#> [1] 302.9  
  
w <- dnorm((area - 120) / 10); w <- w / sum(w)      # pesos que somam 1  
head(data.frame(area, preco, peso = round(w, 3))[order(-w), ], 3)  
  
#>   area preco  peso  
#> 62  120   270 0.051  
#> 61  118   294 0.050  
#> 63  122   310 0.050
```

Leia a saída: os mais próximos pesam $\approx 5\%$ cada — nenhum domina; os 18 a até uma banda (110 a 130 m²) somam 76% do peso. A predição, 302,9, fica perto da do KNN (300,8) e da verdade (297,3).

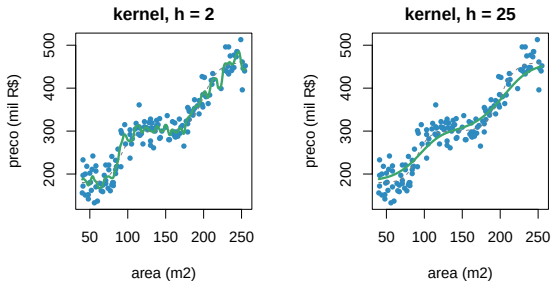
O botão h : um sino estreito ou um largo?

A conta da página anterior em cada ponto da grade — com $h = 2$ e com $h = 25$:

```
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
for (h in c(2, 25)) {
  plot(area, preco, pch = 16, col = "blue", cex = 0.8,
       main = paste("kernel, h =", h),
       xlab = "area (m2)", ylab = "preco (mil R$)")
  lines(grade, f(grade), col = "gray50", lty = 2)
  lines(grade, sapply(grade, nw, x = area, y = preco, h = h),
       col = "green", lwd = 2.5)
}
```

Distância até a verdade — $h = 2$: 13,4; $h = 4$: 8,7; $h = 6$: 7,0; $h = 10$: **6,1**; $h = 15$: 7,2; $h = 25$: 15,0. O melhor KNN tinha chegado a 6,9.

h pequeno treme; h grande achata



Leia a figura: com $h = 2$ o sino alcança de fato só uns seis imóveis, e a curva treme com o ruído deles (variância alta). Com $h = 25$ ele cobre quase 100 m²: os degraus viram rampas e as pontas achatam (viés alto).

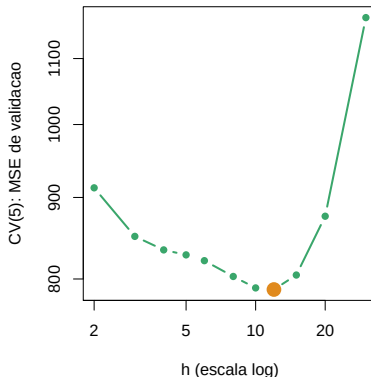
Observação

É o dilema do k , com o h no lugar — e com uma diferença visível: mesmo tremendo, a curva do kernel é *contínua*. O KNN anda em degraus porque cada vizinho entra ou sai de uma vez; no kernel, o peso de cada um muda aos poucos.

Escolhendo h por validação cruzada

```
cv_nw <- function(h) mean(sapply(1:5, function(j) {  
  tr <- dobra != j; va <- dobra == j          # as mesmas 5 dobras  
  pred <- sapply(area[va], nw, x = area[tr], y = preco[tr], h = h)  
  mse(pred, preco[va])  
}))  
hs <- c(2, 3, 4, 5, 6, 8, 10, 12, 15, 20, 30)  
cvh <- sapply(hs, cv_nw)  
setNames(round(cvh), hs)                # CV(5) por h  
  
#>      2      3      4      5      6      8     10     12     15     20     30  
#>  913   851   834   828   821   803   790   788   805   876  1167
```

```
par(mar = c(4,4,1,1), pty = "s")  
plot(hs, cvh, type = "b", log = "xy", pch = 16, col = cverde, lwd = 2,  
      xlab = "h (escala log)", ylab = "CV(5): MSE de validacao")  
abline(h = 625, lty = 2, col = "gray50")          # piso  
points(12, cvh[hs == 12], pch = 16, cex = 2, col = claranja)
```



Leia: o mesmo U, de novo espelhado — h pequeno é o modelo flexível. O fundo é $h = 12$ (CV = 788), com $h = 10$ (790) empatado; o KNN chegou a 785 com $k = 15$, e o polinômio de grau 6 a 788. **Em validação cruzada, empatam todos.** A vantagem dos suavizadores não está nesse número: está em não precisar adivinhar a forma.

KNN × kernel: janela que se adapta × janela fixa

	KNN-regressão	kernel
quem conta	os k mais próximos, com peso igual	todos, com o peso do sino
largura da janela	<i>varia</i> : a que for preciso para juntar k	<i>fixa</i> : h
onde os dados rareiam	a janela alarga e pega vizinhos longe (viés)	sobram poucos vizinhos de fato (variância)
a curva	em degraus	contínua
o botão (maior = mais rígido)	k	h
nesta base, por validação cruzada	$k = 15$: 785	$h = 12$: 788

Em palavras: aqui as áreas estão espalhadas por igual, e os dois empatam. A diferença aparece quando o preditor é *assimétrico* — renda, faturamento, área de galpões: onde os dados rareiam, o KNN vai buscar vizinhos longe, e o kernel fica com poucos — e, longe de todos, os pesos chegam a zero e a conta dá NaN.

Observação

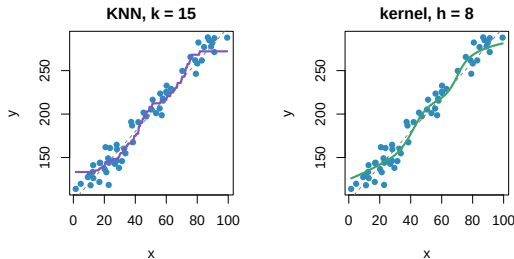
No R base, `ksmooth(x, y, kernel = "normal", bandwidth = b)` faz essa conta — mas **bandwidth não é o h** : o R mede a banda pelos quartis do núcleo, e $h \approx 0,37 \times \text{bandwidth}$. Leia a documentação antes de comparar números.

Onde a média local erra de propósito: a borda

KNN e kernel calculam uma *média* na vizinhança. Numa **rampa**, os vizinhos à esquerda de x_0 puxam para baixo e os à direita puxam para cima; só se compensam se houver vizinhos **dos dois lados**. Na borda, só há de um lado. Uma reta pura deixa isso à mostra:

```
set.seed(3)
xb <- sort(runif(60, 0, 100)); yb <- 100 + 2*xb + rnorm(60, 0, 12)
gb <- seq(min(xb), max(xb), length = 200)
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
plot(xb, yb, pch = 16, col = "blue", main = "KNN, k = 15", xlab = "x", ylab = "y")
abline(100, 2, col = "gray50", lty = 2) # a verdade
lines(gb, sapply(gb, knn_reg, x = xb, y = yb, k = 15), col = "red", lwd = 2.5)
plot(xb, yb, pch = 16, col = "blue", main = "kernel, h = 8", xlab = "x", ylab = "y")
abline(100, 2, col = "gray50", lty = 2)
lines(gb, sapply(gb, nw, x = xb, y = yb, h = 8), col = "green", lwd = 2.5)
```

A média local achata nas pontas — com qualquer peso



Leia a figura: a verdade é $y = 100 + 2x$. Nas pontas, o KNN prevê 133 e 272, e o kernel 126 e 281 (verdade ≈ 103 e ≈ 299): ali os vizinhos estão todos do mesmo lado, e a média fica *dentro* da rampa. O kernel erra um pouco menos, mas do mesmo jeito — trocar o peso não muda a conta.

Observação

A correção é trocar a média por uma **reta local**, ajustada em cada vizinhança: é o **loess**, a curva do `geom_smooth()` do `ggplot` (com menos de 1000 observações) e do `scatter.smooth()`. Os *splines* da Aula 10 também acompanham a tendência até a ponta.

A maldição da dimensionalidade

Por que tudo hoje foi com *um* preditor

Com p preditores, “vizinho” é quem está perto em **todas** as coordenadas ao mesmo tempo — e quase ninguém está. A conta: dados uniformes no cubo $[0, 1]^p$; para uma vizinhança em forma de cubo capturar 10% das observações, o lado dela precisa ser $0,1^{1/p}$:

p	1	2	5	10	20	50
lado da vizinhança	0,10	0,32	0,63	0,79	0,89	0,95

Leia a tabela: com 10 preditores, os “10% mais próximos” cobrem 79% de cada eixo — isso já não é *local*; é quase a média geral. Visto de outro ângulo: para manter 10 vizinhos numa caixa de lado 0,1, precisa-se de $n \approx 10/0,1^p$ observações: 10^3 com $p = 2$, 10^6 com $p = 5$, 10^{11} com $p = 10$.

Observação

Maldição da dimensionalidade: KNN e kernel são excelentes com 1–2 preditores e degradam rápido depois. Duas saídas, que são o resto do Bloco III: **impor estrutura** — um suavizador por preditor, somados (o GAM da Aula 10) — ou métodos que **escolhem onde olhar** (árvores, Aula 11).

Exercícios e trabalho

Exercício

Sete imóveis: áreas (50, 55, 62, 70, 80, 96, 110) e preços (150, 170, 160, 210, 230, 300, 320). Preveja o preço de um imóvel de 74 m² por KNN com $k = 1, 3, 5$ e 7. Para onde a predição caminha quando k cresce?

Exercício

Com os imóveis (62, 160), (70, 210), (80, 230), (96, 300) e o núcleo gaussiano com $h = 10$, calcule os pesos em $x_0 = 74$, normalize-os e obtenha a predição de Nadaraya-Watson. Repita com $h = 3$: quem ainda pesa?

Exercício

Uma validação cruzada deu, para o KNN: $k = 2 \Rightarrow CV = 1300$; $k = 5 \Rightarrow 950$; $k = 10 \Rightarrow 900$; $k = 20 \Rightarrow 940$; $k = 50 \Rightarrow 1500$. Qual k minimiza? Com erro padrão 60 no fundo, qual k a regra de *um erro padrão* (Aula 8) escolhe — e por que aqui “mais simples” é k maior?

Exercício

Com $p = 3$ e com $p = 8$ preditores, que lado precisa ter a vizinhança que captura 10% dos dados? E quantas observações mantêm 10 vizinhos numa caixa de lado 0,1 quando $p = 4$?

Exercícios — conceituais

Exercício (k e a flexibilidade)

Explique, com viés e variância, por que k *pequeno* é o modelo flexível. Qual é o KNN mais rígido possível, e o que ele prevê para qualquer imóvel?

Exercício (A borda)

Numa relação crescente, por que KNN e kernel preveem *de menos* na ponta direita e *de mais* na esquerda? Trocar um pelo outro resolve? O que resolveria?

Exercício (Qual responde)

A diretoria quer “o efeito de 1 m² a mais no preço”. Qual método responde, a reta da Aula 4 ou o kernel? E se ela quer o preço de um imóvel de 300 m², maior que todos da base?

Exercício (Muitos preditores)

Um colega aplicou KNN com 40 preditores e disse que “não deu certo”. Explique por quê e proponha dois caminhos — um deles usando o que a Aula 8 ensinou.

Laboratório — no seu banco de dados

Troque x_1 e y pelo preditor numérico mais importante e pela resposta do seu banco. Reta, KNN e kernel, pelas mesmas dobras:

```
x <- dados$x1; y <- dados$y
ok <- !is.na(x) & !is.na(y); x <- x[ok]; y <- y[ok] # sem NA
knn_reg <- function(x0, x, y, k) mean(y[order(abs(x - x0))[1:k]])
nw <- function(x0, x, y, h) { w <- dnorm((x - x0) / h); sum(w * y) / sum(w) }
reta <- function(x0, x, y) sum(coef(lm(y ~ x)) * c(1, x0))
set.seed(1); dobra <- sample(rep(1:5, length = length(y)))
cv_de <- function(prever) mean(sapply(1:5, function(j) { # CV(5) de um metodo
  tr <- dobra != j; va <- dobra == j
  mean((y[va] - sapply(x[va], prever, x = x[tr], y = y[tr]))^2)
}))
cv_de(reta) # a reta
ks <- c(3, 5, 10, 20, 40); ks <- ks[ks < 0.8 * length(y)] # KNN
sapply(ks, function(k) cv_de(function(x0, x, y) knn_reg(x0, x, y, k)))
hs <- sd(x) * c(0.05, 0.1, 0.2, 0.4) # kernel
sapply(hs, function(h) cv_de(function(x0, x, y) nw(x0, x, y, h)))
g <- seq(min(x), max(x), length = 200) # a figura
plot(x, y, col = "gray60"); lines(g, sapply(g, nw, x = x, y = y, h = hs[2]), lwd = 2)
```

Entregar (código em estrutura/codigo/, registro em consolidados/): a figura com o melhor h ; o k e o h escolhidos; o erro de CV da reta, do KNN e do kernel; e *uma frase* dizendo **onde** a relação não é uma reta — e, se for, fique com a reta. Deu NaN no kernel? É a janela fixa sem vizinhos: aumente h .

- **Suavizar** é prever em x_0 olhando só a vizinhança de x_0 : sem supor forma, supondo *suavidade*.
- **KNN-regressão**: média dos k mais próximos. **Kernel**: média de todos, ponderada por um sino de largura h — a mesma ideia, com curva contínua.
- O botão (k ou h) sai da **validação cruzada**; nos dois, maior = mais rígido.
- Toda média local **achata nas pontas**; e, com muitos preditores, “vizinho” deixa de existir: a **maldição da dimensionalidade**.

Próxima aula: suavizar por pedaços e com controle — *splines*, que acompanham a tendência até a ponta — e somar um suavizador por preditor: os **modelos aditivos** (GAM), a resposta à maldição.